Amr Ahmed Abdelal

Date: __/__/20 71588 Day: _____

Q1] Algorithm Design and Asymptotic Analysis

1- Algorithmic strategy:

The algorithm uses a Linear scanning strategy. We start by considering the first element as both the largest and smallest element. Then we scan the remaining elements one by one. if an element is greater then the current largest value we update the largest value. if an element is smaller then the current smallest value. we update the smallest value.

2- Pseudocode:

Algorithm findminmax(A,n)

Input:

An array A containing n integers

Output:

The largest and smallest elements

```
1- largest ← A[0]
2- smallest ← A[0]
3- For i←1 To n-1 do
4- If A[i] > largest then
5- largest ← A[i]
6- end if
7- if A[i] < smallest then
8- smallest ← A[i]
9- end if
10- end for
11- return largest, smallest (1)
End Algorithm
```

Amr Ahmed Abdelal

Date: / /20 71588 Day:

3- major Primitive Operations

The major Primitive Operations are:

1- Comparison: A[i] > Largest

2- Comparison: A[i] < Smallest

3- Assignment: Largest ← A[i]

4- Assignment: Smallest ← A[i]

5- Loop Iteration.

The main operation used for Time analysis is Comparison

4- Best-case and worst-case Running Time

The loop runs from 1 to n-1, so it performs n-1 iterations

in every iterations, Two comparisons are performed

Therefore:

$T(n) = 2(n-1)$

Best-case:

$T_{best}(n) = 2(n-1)$

worst-case:

$T_{worst}(n) = 2(n-1)$

5- Asymptotic Notation:

Best case = $\Omega(n)$

worst case = $O(n)$

Overall = $\Theta(n)$

Therefore, The running time of the algorithm is:

$\Theta(n)$

6- Space Complexity:

The algorithm uses only a constant number of extra variables such as Largest, smallest and i.

Therefore: Space Complexity = $O(1)$

Ali Ahmed Algelal

Date / /20 71588 Day:

Q2) Design and compare Algorithms:

1- Linear search

1- Algorithmic strategy:

Linear search checks the elements of the array one by one from the beginning. If the value X is found the algorithm return true if all elements are checked and X is not found, it returns false.

2- Pseudocode:

Algorithm LinearSearch(A, n, X)

Input:

An array A containing n integers

A value X To search For

output:

True if X exists, otherwise False

1- For i <- 0 To n-1 do

2- if A[i] = X then

3- return True

4- end if

5- end for

6- return False

end Algorithm

3- Best case and worst case time complexity

Best case:

if x is the first element

T best(n) = 1

Therefore,

Best case = $\Omega(1)$

CamScanner

Ahmed Abdal

Date: / /20 7 1588 Day:

WorstCase:
if x is the last element or does not exist, all n elements must be checked
Tworst(n) = n
Therefore:
worst case = O(n)
1- Big-O, Big-Ω and Big-Θ
Best case = Ω(1)
worst case = O(n)
Average case = Θ(n)

2- Binary search:
1- Algorithmic Strategy
Binary search works on a sorted array. It compares x with the middle element if x is equal to the middle element, it is found if x is smaller, we search the the left half, if x is larger, we search the right half
* Therefore, approximately half of the search space is eliminated in every step
2- Pseudocode
Algorithm BinarySearch(A, n, x)

Input:
A sorted array A containing n integers
A value x to search for
Output:
true if x exists, otherwise false

1- Low=0
2- High=n-1
3- while Low ≤ High do
4- mid=Floor((Low+High)/2)
5- if A(mid) = x Then
6- return True
7- else if x<A(mid)
8- High=mid-1
9- else
10- Low=mid+1
11- end if
12- end while
13- return False

Amr Ahmed Abdelal

Date: / /20 21588 Day:

3- Best-case and worst-case Time Complexity:

Best Case:

if x is the middle element in the first comparison

Best case = $\Omega(1)$

Worst Case:

The search space is divided by two after every comparison.

Therefore

Worst Case = $O(\log n)$

4- Big-O, Big-Ω and Big-Θ

Best case = $\Omega(1)$

worst case = $O(\log n)$

Average Case = $\Theta(\log n)$

II- Comparison of the Two Algorithms

	Linear search	Binary search
Array	Any array	sorted array
Best Case	$\Omega(1)$	$\Omega(1)$
worst case	$O(n)$	$O(\log n)$
Average case	$\Theta(n)$	$\Theta(\log n)$
Method	check elements one by one	divide array into halves
space	$O(1)$	$O(1)$

5. Conclusion:

Binary search is more efficient for large sorted arrays because its worst-case time complexity is $O(\log n)$, while Linear search has a worst-case time complexity of $O(n)$

CamScanner

Amr Ahmed Abdelal

Date: / /20 #1588 Day:

12- Effect of Input size and Arrangement:

The choice of algorithm depends on the size and arrangement of the input data for a small array, Linear search can be simple and efficient for a large sorted array, Binary search is more efficient because it reduces the search space by half after each comparison.

if the array not sorted, Binary search cannot be directly applied, so Linear search is more appropriate.

Therefore:

unsorted Array → Linear search

sorted Array → Binary search

large sorted Array → Binary search

Final conclusion:

Linear search → O(n)

Binarysearch → O(log n)